Part 3 — Benchmark Results

David Londres — CIn/UFPE

2026-06-06

Context

Phase 3 measures the latency distributions that constitute the thesis's primary deliverable. Both Phase 2 pipelines — **FastAPI + ONNX Runtime** and **BentoML + adaptive batching** — were driven with open-loop Vegeta load (per the proposal's anti-Coordinated-Omission requirement). Two deliverables: a **matrix sweep** of P50 / P99 / P99.9 across (workers × ORT threads) × RPS, and a **latency budget** decomposing each pipeline's latency into stack layers (L1 inference / L2 framework+HTTP / L3 containerised).

To compare a load-shedding system (BentoML's dispatcher returns 503 when its SLA deadline can't be met) fairly against a queueing system (FastAPI absorbs overload as latency), the report leads with a **goodput-within-SLA** metric: the fraction of offered load served within 50 ms with a 2xx response.

Method

Three measurement layers, all routed through HTTP so BentoML's batching is exercised the same way wherever it applies.

Layer	Isolates	How
L1	Pure ONNX cost	`session.run` in a tight Python loop, no HTTP.
L2	Framework + parsing + TCP loopback	Server on host with `taskset -c 0,1`; Vegeta hits it directly. (1 worker, 2 ORT threads) @ 150 RPS, 45 s.

Layer	Isolates	How
L3	Full deployment	Server in Docker on `--cpuset-cpus=0,1`; native Vegeta on cores 2–3. Matrix: pipelines × (workers, threads) in {(1,1),(1,2),(2,1),(2,2)} × RPS in {50, 150, 350, 600, 1000} × 3 repeats × 45 s.

BentoML's `max_latency_ms` is promoted to an explicit axis (variants `lat5`, `lat50`, `lat250` ms; `max_batch_size = 16` fixed). The full sweep ran in **197 minutes** on WSL2 Ubuntu, producing **219 cells × 3 repeats** with **0 failures**.

Inference floor (L1, 10 000 iterations, hot cache):

batch	per-call P50	per-row P50
1	**0.014 ms**	0.014 ms
16	0.349 ms	0.022 ms

The model itself costs ~14 µs per single-row inference and is therefore ~1 % of any served request's total latency. Everything else the report measures is *serving-stack overhead*.

Result 1 — Goodput within a 50 ms SLA (the fairness-corrected headline)

FastAPI sustains 99.7–100 % goodput from 50 to 1000 RPS in every parallelism cell. BentoML's three variants are nearly indistinguishable up to ~150 RPS, then diverge: each variant has a cliff where goodput collapses, and **the cliff moves with worker count, not with thread count**. The (1, *) cells collapse around 600 RPS; the (2, *) cells hold to ~1000 RPS — exactly the fingerprint of a per-worker bottleneck (the dispatcher coroutine — see Mechanism below). At moderate load (<= 350 RPS) all variants behave comparably; the deadline knob only changes the *shape* of the failure (active 503-shedding at low values, passive queue-then-timeout at high values).

Fraction of offered load served within 50 ms with a 2xx response, per parallelism cell and BentoML deadline variant. FastAPI holds at or near 100 % across the matrix; every BentoML variant drops to ~10 % goodput somewhere between 350 RPS and 1000 RPS.

Figure 1: Fraction of offered load served within 50 ms with a 2xx response, per parallelism cell and BentoML deadline variant. FastAPI holds at or near 100 % across the matrix; every BentoML variant drops to ~10 % goodput somewhere between 350 RPS and 1000 RPS.

P99 latency vs RPS per parallelism cell (log y-axis). FastAPI's P99 is essentially flat near 2–3 ms across the matrix; BentoML's P99 climbs by two orders of magnitude across the same RPS range.

Figure 2: P99 latency vs RPS per parallelism cell (log y-axis). FastAPI's P99 is essentially flat near 2–3 ms across the matrix; BentoML's P99 climbs by two orders of magnitude across the same RPS range.

Tail variability per parallelism cell (log y-axis). FastAPI's jitter ratio stays in the 2–4× range; BentoML reaches 100–300× under load.

Figure 3: Tail variability per parallelism cell (log y-axis). FastAPI's jitter ratio stays in the 2–4× range; BentoML reaches 100–300× under load.

Result 2 — P99 inflection (the proposal's stated headline metric)

The shape difference is what the proposal's "P99 tail latency" headline is about. FastAPI's P99 is ~1.7 ms at 150 RPS and ~1.7 ms at 600 RPS — flat. BentoML's P99 at the same (1, 2) cell goes $5 \rightarrow 70 \rightarrow 200 \rightarrow > 3000$ ms across 50 / 150 / 350 / 600 RPS. The dispatcher's queue depth controls P99 almost directly: small at low RPS, exponentially-growing once the dispatcher's serial drain rate is exceeded.

Result 3 — Jitter (P99 / P50)

The proposal calls out "jitter compression" — the ability to keep P99 close to P50 — as a key serving-stack property. FastAPI's P99 / P50 ratio stays in the 2–4× band across the matrix even at 1000 RPS. BentoML's ratio climbs from ~3× at low load to 30 × at 350 RPS and crosses 100× at saturation. Adaptive batching does not compress jitter for this workload — it amplifies it, because the dispatcher's queue introduces variable per-request wait.

Result 4 — Latency budget at (1 worker, 2 threads), 150 RPS

The L1 bar (~14 µs) is the model's intrinsic cost — identical across all variants because they all run the same `model.onnx`. What separates the pipelines is the L2 bar (framework + HTTP, no Docker): FastAPI at 0.6 ms, BentoML lat50/lat250 at ~1.5–1.6 ms, BentoML lat5 at 6.7 ms (the tight deadline is already in trouble at 150 RPS even without containerisation). The L3 bar (full deployment) adds a small near-constant Docker overhead. **Almost all measured latency is serving-stack overhead, not inference.**

Per-variant P50 across the three measurement layers. The model floor (L1) is the same ~0.014 ms across every variant; what differs is the framework / dispatcher cost between L1 and L2.

Figure 4: Per-variant P50 across the three measurement layers. The model floor (L1) is the same ~0.014 ms across every variant; what differs is the framework / dispatcher cost between L1 and L2.

Mechanism — Why BentoML collapses despite being designed for concurrency

The collapse RPS in Result 1 scales with worker count and not with thread count. That is the fingerprint of a single-coroutine bottleneck per worker, which matches BentoML's architecture: the `controller()` coroutine in dispatcher.py is a *standalone asyncio coroutine* that drains the queue serially, and every request — every dispatch decision, every batch assembly — funnels through it. Inference itself parallelises fine (ONNX Runtime releases the GIL inside `session.run`), but the plumbing around inference is serialised. At high RPS the queue grows faster than the coroutine can drain it, so requests either exceed `max_latency_ms` (→ 503) or exceed Vegeta's 3 s cap (→ timeout). Doubling workers doubles dispatcher throughput — hence the 2-worker cells holding higher RPS.

Adaptive batching is designed to amortise **fixed per-call overhead** (GPU kernel launches, deep-net startup). For our 14 µs XGBoost ONNX model, there is essentially no fixed overhead to amortise — batching 16 rows saves about −7 µs/row at the cost of ~100–1000 µs of dispatcher work. The trade inverts. The result is expected to flip for workloads with non-trivial fixed per-call cost (deep networks, GPU execution).

Conclusion

For cheap single-row tree-ensemble inference on CPU, **FastAPI + ONNX Runtime sustains 99.7–100 % goodput within a 50 ms SLA across the entire parallelism × RPS matrix**, with P50 0.7–1.5 ms and P99 / P50 jitter in the 2–4× band. BentoML + adaptive batching trails FastAPI at every cell and every RPS; its dispatcher saturates around the per-worker collapse RPS, where P99 grows by two orders of magnitude. Adaptive batching adds no measurable benefit at this scale because the inference is too cheap to amortise.

Artifacts

- **Data**: `project/3-bench/results/{layer1, layer2, layer3}/`, `summary.csv` (219 runs × all percentiles + goodput@SLA), `budget.csv`.
- **Figures**: `project/3-bench/results/figures/` — embedded above.
- **Detailed write-up**: `thesis/phase3.md`.